1.a

$$\begin{aligned}
p(y;\lambda) &= \frac{e^{-\lambda}\lambda^y}{y!} \\
&= \frac{1}{y!}e^{-\lambda}e^{\log(\lambda^y)} \\
&= \frac{1}{y!}e^{-\lambda}e^{y\log(\lambda)} \\
&= \frac{1}{y!}e^{y\log(\lambda)-\lambda}
\end{aligned}$$

Now, we can define the base measure, natural parameter, sufficient statistic, and log partition function.

$$\begin{aligned}
\eta &= \log(\lambda) \\
T(y) &= y \\
a(\eta) &= \lambda \\
&= e^{\eta} \\
b(y) &= \frac{1}{y!}
\end{aligned}$$

1.b

$$\begin{aligned} g(\eta) &= E[T(y);\eta] \\ &= E[y \mid x;\theta] \\ &= \lambda \\ &= e^{\eta} \\ &= e^{\theta^T x} \end{aligned}$$

1.c

The log-likelihood of an example $(x^{(i)}, y^{(i)})$ is defined as $\ell(\theta) = \log p(y^{(i)}|x^{(i)};\theta)$. To derive the stochastic gradient ascent rule, use the results in part (a) and the standard GLM assumption that $\eta = \theta^T x$.

$$
\begin{aligned}
\frac{\partial \ell(\theta)}{\partial \theta_j} &= \frac{\partial \log p(y^{(i)}|x^{(i)};\theta)}{\partial \theta_j} \\
&= \frac{\partial \log\left(\frac{1}{y^{(i)}!}\exp(\eta^T y^{(i)} - e^{\eta})\right)}{\partial \theta_j} \\
&= \frac{\partial}{\partial \theta_j}\left(\log(\frac{1}{y^{(i)}!}) + \log(e^{\theta^T x^{(i)} y^{(i)} - e^{\theta^T x^{(i)}}})\right) \\
&= \frac{\partial}{\partial \theta_j}\left(-\log(y^{(i)}!) + \theta^T x^{(i)} y^{(i)} - e^{\theta^T x^{(i)}}\right) \\
&= x^{(i)} y^{(i)} - x^{(i)} e^{\theta^T x^{(i)}} \\
&= x^{(i)} y^{(i)} - x^{(i)} e^{\theta^T x^{(i)}} \\
&= x^{(i)}\left(y^{(i)} - e^{\theta^T x^{(i)}}\right)
\end{aligned}
$$

Thus the stochastic gradient ascent update rule should be:

$$\theta_j := \theta_j + \alpha \frac{\partial \ell(\theta)}{\partial \theta_j},$$

which reduces here to:

$$\theta_j := \theta_j + \alpha \cdot x^{(i)}\left(y^{(i)} - e^{\theta^T x^{(i)}}\right)$$

2.a

We know that the definition of positive semi-definite matrices is $\forall x \in \mathbb{R}^d, x^T M x \geq 0$ where $M \in \mathbb{R}^d \times \mathbb{R}^d$.
For any $x \in \mathbb{R}^d$, $x^T K x = x^T (K_1 + K_2) x = x^T K_1 x + x^T K_2 x$.
Since K_1 and K_2 are both kernels, they are both positive semidefinite matrices. Therefore, $x^T K_1 x \geq 0$ and $x^T K_2 x \geq 0$. This means that $x^T K x = x^T K_1 x + x^T K_2 x \geq 0$ and that K is a positive semidefinite matrix. This implies that the matrix is symmetric. Therefore, $K(x, z)$ is necessarily a kernel.

2.b

This is not true. For example,

$$K_1\left(\left[\begin{array}{c}1\\2\end{array}\right],\left[\begin{array}{c}3\\-1\end{array}\right]\right)=\left[\begin{array}{cc}K_1(x,x) & K_1(x,z)\\K_1(z,x) & K_1(z,z)\end{array}\right]=\left[\begin{array}{cc}x^Tx & x^Tz\\z^Tx & z^Tz\end{array}\right]=\left[\begin{array}{cc}5 & 1\\1 & 10\end{array}\right]$$

$$|\lambda I-K_1|=\left|\begin{array}{cc}\lambda-5 & -1\\-1 & \lambda-10\end{array}\right|=(\lambda-5)(\lambda-10)-(-1)(-1)=\lambda^2-15\lambda+49$$

$$\lambda^2-15\lambda+49=0\Rightarrow\lambda=\frac{15\pm\sqrt{29}}{2}\approx 9.19, 5.81$$

$$K_2\left(\left[\begin{array}{c}1\\2\end{array}\right],\left[\begin{array}{c}3\\-1\end{array}\right]\right)=\left[\begin{array}{cc}K_2(x,x) & K_2(x,z)\\K_2(z,x) & K_2(z,z)\end{array}\right]=\left[\begin{array}{cc}(x^Tx+1)^2 & (x^Tz+1)^2\\(z^Tx+1)^2 & (z^Tz+1)^2\end{array}\right]=\left[\begin{array}{cc}36 & 4\\4 & 121\end{array}\right]$$

$$|\lambda I-K_2|=\left|\begin{array}{cc}\lambda-36 & -4\\-4 & \lambda-121\end{array}\right|=(\lambda-36)(\lambda-121)-(-4)(-4)=\lambda^2-157\lambda+4340$$

$$\lambda^2-157\lambda+4340=0\Rightarrow\lambda=\frac{157\pm\sqrt{7289}}{2}\approx 121.2, 35.8$$

$$K(x,z)=K_1(x,z))+K_2(x,z)=\left[\begin{array}{cc}5 & 1\\1 & 10\end{array}\right]-\left[\begin{array}{cc}36 & 4\\4 & 121\end{array}\right]=\left[\begin{array}{cc}-31 & -3\\-3 & -111\end{array}\right]$$

$$|\lambda I-K|=\left|\begin{array}{cc}\lambda+31 & 3\\3 & \lambda+111\end{array}\right|=(\lambda+31)(\lambda+111)-(3)(3)=\lambda^2+142\lambda+3432$$

$$\lambda^2+142\lambda+3432=0\Rightarrow\lambda=\frac{-142\pm\sqrt{6436}}{2}\approx -30.875, -111.125$$

Since the eigenvalues of $K(x,z)$ are negative, $K(x,z)$ is not positive semidefinite, so it cannot be a kernel.

2.c

We know that the definition of positive semi-definite matrices is $\forall x \in \mathbb{R}^d, x^T M x \geq 0$ where $M \in \mathbb{R}^d \times \mathbb{R}^d$.
For any $x \in \mathbb{R}^d$, $x^T K x = x^T (aK_1) x = a x^T K_1 x$.
Since $K_1$is a kernel, it is a positive semidefinite matrix. Therefore, $x^T K_1 x \geq 0$. We know that $a \in \mathbb{R}^+$ so $x^T K x = a x^T K_1 x \geq 0$. This means that K is a positive semidefinite matrix. This implies that the matrix is symmetric. Therefore, $K(x, z)$ is necessarily a kernel.

2.d

This is not true. For example, if $-a = -1$

$$K_1\left(\left[\begin{array}{c}1\\2\end{array}\right],\left[\begin{array}{c}3\\-1\end{array}\right]\right) = \left[\begin{array}{cc}K_1(x,x) & K_1(x,z)\\ K_1(z,x) & K_1(z,z)\end{array}\right] = \left[\begin{array}{cc}x^Tx & x^Tz\\ z^Tx & z^Tz\end{array}\right] = \left[\begin{array}{cc}5 & 1\\ 1 & 10\end{array}\right]$$

$$|\lambda I - K_1| = \left|\begin{array}{cc}\lambda-5 & -1\\ -1 & \lambda-10\end{array}\right| = (\lambda-5)(\lambda-10)-(-1)(-1) = \lambda^2 - 15\lambda + 49$$

$$\lambda^2 - 15\lambda + 49 = 0 \Rightarrow \lambda = \frac{15 \pm \sqrt{29}}{2} \approx 9.19, 5.81$$

$$K\left(\left[\begin{array}{c}1\\2\end{array}\right],\left[\begin{array}{c}3\\-1\end{array}\right]\right) = -1 \cdot K_1\left(\left[\begin{array}{c}1\\2\end{array}\right],\left[\begin{array}{c}3\\-1\end{array}\right]\right) = \left[\begin{array}{cc}-5 & -1\\ -1 & -10\end{array}\right]$$

$$|\lambda I - K| = \left|\begin{array}{cc}\lambda+5 & 1\\ 1 & \lambda+10\end{array}\right| = (\lambda+5)(\lambda+10)-(1)(1) = \lambda^2 + 15\lambda + 49$$

$$\lambda^2 + 15\lambda + 49 = 0 \Rightarrow \lambda = \frac{-15 \pm \sqrt{29}}{2} \approx -4.8075, -10.1925$$

Since the eigenvalues of $K(x,z)$ are negative, $K(x,z)$ is not positive semidefinite, so it cannot be a kernel.

3.ai

We are given that the update rule for this version of the perceptron algorithm is given by

$$\theta^{(i+1)} := \theta^{(i)} + \alpha(y^{(i+1)} - h_{\theta^{(i)}}(x^{(i+1)}))x^{(i+1)}$$

We start with the beginning time step as

$$\theta^{(0)} = \vec{0}$$

We will demonstrate how the vector θ can be represented as an implicit linear combination of feature vectors using an induction proof.

- Base Case at Iteration 0: At iteration 0, we initialize:

$$\theta^{(0)} = 0 = \sum_{i=1}^{n} 0 \cdot \phi(x^{(i)})$$

  Here, all coefficients $\beta_i^{(0)}$ are zero, which affirms that θ is indeed a linear combination of the feature vectors, as required.

- Step for Iteration 1: At iteration 1, we update θ according to:

$$\begin{aligned}
\theta^{(1)} &:= \theta^{(0)} + \alpha \sum_{i=1}^{n} \left(y^{(i)} - \mathsf{sign}({\theta^{(0)}}^T \phi(x^{(i)})) \right) \phi(x^{(i)}) \\
&= \vec{0} + \alpha \sum_{i=1}^{n} \left(y^{(i)} - 1 \right) \phi(x^{(i)}) \\
&= \sum_{i=1}^{n} \quad \underbrace{\alpha(y^{(i)} - 1)}_{\beta_i \text{ at iteration 1}} \quad \phi(x^{(i)})
\end{aligned}$$

  This establishes that $\theta^{(1)}$ is a linear combination of the feature vectors.

- Induction Hypothesis: Assume at iteration t, the representation holds:

$$\theta = \sum_{i=1}^{n} \beta_i \phi(x^{(i)})$$

- Induction Step at Iteration $t+1$: Show this holds for iteration $t+1$ to finish the induction proof:

$$\begin{aligned}
\theta^{(t+1)} &:= \theta^{(t)} + \alpha \sum_{i=1}^{n} (y^{(i)} - h_{\theta^{(t)}}(x^{(i)}))x^{(i)} \\
&= \sum_{i=1}^{n} \beta_i \phi(x^{(i)}) + \alpha \sum_{i=1}^{n} \left(y^{(i)} - \mathsf{sign}({\theta^{(t)}}^T \phi(x^{(i)})) \right) \phi(x^{(i)}) \\
&= \sum_{i=1}^{n} \quad \underbrace{(\beta_i + \alpha \left(y^{(i)} - \mathsf{sign}({\theta^{(t)}}^T \phi(x^{(i)})) \right))}_{\text{new } \beta_i} \quad \phi(x^{(i)})
\end{aligned}$$

  We have shown that at iteration $t+1$, θ remains a linear combination of the feature vectors with updated coefficients β_i.

By induction, we showed that at each iteration, θ can be represented as:

$$\theta = \sum_{i=1}^{n} \beta_i \phi(x^{(i)})$$

for some $\beta_i \in \mathbb{R}^d$ where the update rule for β is

$$\beta_i := \beta_i + \alpha \left(y^{(i)} - \mathsf{sign}(\theta^T \phi(x^{(i)})) \right)$$

Replacing θ^T with $\sum_{j=1}^{n} \beta_j \phi(x^{(j)})$, we get:

$$\forall i \in \{1, \ldots, n\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \text{sign}(\sum_{j=1}^{n} \beta_j \langle \phi(x^{(j)}, \phi(x^{(i)}) \rangle) \right)$$

Now the Kernel trick is to define a Kernel corresponding to the feature map ϕ as a function that maps $\mathcal{X} \times \mathcal{X} \to \mathbb{R}$ satisfying:

$$K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle$$

With K being an $n \times n$ matrix with $K_{ij} = K(x^{(i)}, y^{(i)})$, this means that we can simplify our β_i update rule to

$$\beta := \beta + \alpha(\vec{y} - K\beta)$$

3.aii

Given a new input $x^{(i+1)}$, we calculate the hypothesis prediction the perceptron gives by doing the following:

$$\begin{aligned} h_{\theta^{(i)}}(x^{(i+1)}) &= \text{sign}({\theta^{(i)}}^T \phi(x^{(i+1)})) \\ &= \text{sign} \left[\left(\sum_{j=1}^{i} \beta_j \phi(x^{(j)}) \right)^T \phi(x^{(i+1)}) \right] \\ &= \text{sign} \left(\sum_{j=1}^{i} \beta_j K(x^j, x^{(i+1)}) \right) \end{aligned}$$

Here, $K(x, z) = \langle \phi(x), \phi(z) \rangle$. We don't need to know the function $\phi(x)$. The kernel function acts as a proxy for the inner product of ϕ.

3.aiii

To perform an update to θ using a new training example $(x^{(i+1)}, y^{(i+1)})$, we can redefine the update rule as follows. We know what $\theta^{(i)}, y^{(i+1)}, \alpha$ is. Using $x^{(i+1)}$, we can also calculate what $h_{\theta^{(i)}}(x^{(i+1)})$ is:

$$h_{\theta^{(i)}}(x^{(i+1)}) = \text{sign} \left(\sum_{j=1}^{i} \beta_j K(x^j, x^{(i+1)}) \right)$$

Here, we can calculate what β is using the given values for the previous β update rule, the kernel, $y^{(i+1)}$, and α.

$$\begin{aligned} \theta^{(i+1)} &:= \theta^{(i)} + \alpha(y^{(i+1)} - \text{sign} \left(\sum_{j=1}^{i} \beta_j K(x^j, x^{(i+1)}) \right))\phi(x^{(i+1)}) \\ &= \beta_{i+1} \phi(x^{(i+1)}) \end{aligned}$$

This is the final representation of the update rule to θ, with the following recursive definition for β:

$$\beta_i := \beta_{i-1} + \alpha \left(y^{(i)} - \text{sign}(\sum_{j=1}^{i-1} \beta_j K(x^{(j)}, x^{(i)}) \right)$$

where $\beta_0 = \vec{0}$.

While this is a representation of updating θ, we will never need to update θ directly since the real calculation for the hypothesis can be done without ever referring to ϕ, as seen in 3(aii).

3.c

The dot product kernel performs poorly. This is because the dot product kernel gives a linear estimate (returns a value in the same $\mathbb{R}$ space as the input), while the dataset we are given has a nonlinear boundary in the form of a circle.